CSE 216 Formal Methods
Fall 2026 Daniel Fremont

HW 2

Due October 13 at 3:00 pm

(6 questions, 230 points total)

I worked with vadim pelyushenko on some problems, mainly cross checking answers

1. (20 pts.) Pigeonhole principle

Recall that the pigeonhole principle says that if you have n pigeons and m pigeonholes for them, if $n > m$ then at least one pigeonhole will contain multiple pigeons. Let the variable $x_{i,j}$ indicate that pigeon i is in pigeonhole j, and let $F_{n,m}(x_{1,1},\ldots,x_{n,m})$ be true if and only if the variables encode a valid arrangement of pigeons where each pigeonhole has at most one pigeon.

(a) Give a CNF formula for $F_{n,m}$, describing the meaning of each of the clauses. How large is your formula as a function of n and m?

Answer: there are 3 things we have to check.

i. each hole has at most 1 pigeon
ii. each pigeon is assigned at least one hole
iii. each pigeon is assigned at most one hole

i.
$$\neg x_{i_1,j} \vee \neg x_{i_2,j} \quad \text{for each } j \in \{1,\ldots,m\} \text{ and } 1 \le i_1 < i_2 \le n \tag{1}$$

ii.
$$\bigvee_{j=1}^{m} x_{i,j} \quad \text{for each } i \in \{1,\ldots,n\} \tag{2}$$

iii.
$$\neg x_{i,j_1} \vee \neg x_{i,j_2} \quad \text{for each } i \in \{1,\ldots,n\} \text{ and } 1 \le j_1 < j_2 \le m \tag{3}$$

The for each means an and between the expressions. The total number of expressions here are

i. $2m\binom{n}{2}$
ii. mn
iii. $2n\binom{m}{2}$

the total number of clauses is $O(n^2m + m^2n)$

(b) Asymptotically, how large would the ROBDD (with your choice of variable order) for $F_{n+1,n}$ be? Why?

Answer: its always just $\perp$ since there are too many pigeons and there are no valid assignments no matter what n is.

2. (40 pts.) Counting solutions with a BDD

Design an algorithm to efficiently count the number of satisfying assignments of a Boolean function, given an ROBDD for it.

(Hint: think about the BDD for $x \leftrightarrow y$ first, then $x \vee y$.)

 Assignment © 2026 Daniel Fremont. Redistribution online is prohibited.

(a) Define your algorithm using either pseudocode or an English description that details every step of the algorithm.

Answer: let v set of vertices in the robdd (including ones simplified out of the ROBDD). let v_0 be the number of variables.

Counting the number of satisfying assignments is counting the number of paths the BDD has ending at $\top$ times 2 to the power of the number of unassigned variables in the path.

algorithm:

first, topologically sort the DAG.

create an array $DP[|v|][v_0]$ in size. (DP[a][b] represents the number of paths from the source (root node) to a of length b.)

initialize DP[root][0] = 1.

update in vertices u in topological order, starting from length zero. for each $l \in [v_0]$, for each $u \in v$, DP[v][l+1]=DP[v][l+1] + DP[u][l].

return

$$\sum_{i \in [v]} 2^{v-i} DP[\top][i] \tag{4}$$

(b) Argue why your algorithm is correct.

Answer: if a path has length exactly v and ends at $\top$, then that path corresponds to exactly 1 satisfying assignment. for each "free" variable, in the path, it leads to $2^{\#\text{ of free var}}$ satisfying assignments. the DP just counts paths, and retains the length of the paths. the sum makes sure that the paths actually reflect the number of assignments.

(c) What is the asymptotic runtime of your algorithm?

Answer: assuming the return summation is fast, the runtime of sorting the dag is $O(v)$, the runtime of updating the DP array is $O(|v| \cdot v_0)$

3. (50 pts.) Horn clauses

Recall that a *Horn clause* is a clause with at most one positive literal. For example, $\neg x \vee \neg y \vee z$ and x are Horn clauses, but $x \vee y$ is not.

(a) Give a polynomial-time algorithm to decide whether a set of Horn clauses is satisfiable. *(Hint: view each Horn clause as an implication.)*

Answer: There are two cases. either

i. all clauses have a negated variable

ii. there exists one clause that does not have a negated variable

i. the first case, we can set all variables to false and the total formula is true since each clause has at least one $\top$

ii. in the second case, we can do unit propagation since if there is a clause that does not have a negated variable, it must be by itself.
recurse this process until either all clauses are true, or there is a contradiction in two clauses (unit prop is impossible since one clause is the negation of another clause)

(b) Argue why your algorithm is correct and runs in polynomial time.

Answer: there are at most $O(n)$ unit propogations and there are $O(n)$ clauses to update each unit prop, so the runtime is $O(n^2)$

(c) The Tseitin transformation can turn any Boolean formula into an equisatisfiable CNF formula in polynomial (in fact linear) time. Is there a corresponding efficient transformation of arbitrary formulas into Horn clauses?

Answer: no, (not that we know of) since if there were, $\mathbb{P} = \mathbb{NP}$. if there were efficient transformation of arbitrary cnf into horn, then the composition of the transformation and the poly algo here would mean any cnf has a polytime algorithm for SAT.

4. (20 pts.) Splitting

(a) Consider this recursive SAT solving algorithm: given a CNF, write it as $\phi \wedge \psi$ by splitting the set of clauses in half (i.e. if there are m clauses, letting ϕ be the first $m/2$ of them and ψ be the rest), and recursively check whether both ϕ and ψ are satisfiable. Is this algorithm correct? Why or why not?

Answer: No. each half could return sat but the assignment of the same variable could differ on each half which is not an issue for ϕ and ψ individually, but is a problem for $\phi \wedge \psi$.

(b) It turns out that replacing "satisfiable" with "valid" above gives a correct algorithm for deciding validity that runs in linear time! Since ϕ is satisfiable if and only if $\neg\phi$ is not valid, why doesn't this prove P = NP?

Answer: for validity, the cnf as long as all clauses are a tautology, containing a clause like $x \vee \neg x$, the whole cnf is valid. The assignments could clash between each clause but it causes no problems since any assignment in the final formula should always output $\top$. the problem is that the negation of ϕ is not a cnf.

5. (20 pts.) Simplification

Find a simplified equisatisfiable formula to the one below using unit propagation and pure literal elimination, showing each step separately:

$$z \wedge (x \vee y) \wedge (\neg x \vee y \vee u) \wedge (\neg y \vee \neg w) \wedge (w \vee z) \wedge (\neg u \vee \neg z) \wedge (u \vee s \vee t) \wedge (\neg z \vee \neg s \vee \neg t)$$

Answer:

$$\begin{aligned}
&= z \wedge (x \vee y) \wedge (\neg x \vee y \vee u) \wedge (\neg y \vee \neg w) \wedge (w \vee z) \wedge (\neg u \vee \neg z) \wedge (u \vee s \vee t) \wedge (\neg z \vee \neg s \vee \neg t) \\
&= \top \wedge (x \vee y) \wedge (\neg x \vee y \vee u) \wedge (\neg y \vee \neg w) \wedge (w \vee \top) \wedge (\neg u \vee \neg \top) \wedge (u \vee s \vee t) \wedge (\neg \top \vee \neg s \vee \neg t) \\
&= (x \vee y) \wedge (\neg x \vee y \vee u) \wedge (\neg y \vee \neg w) \wedge (\neg u) \wedge (u \vee s \vee t) \wedge (\neg s \vee \neg t) \\
&= (x \vee y) \wedge (\neg x \vee y \vee \bot) \wedge (\neg y \vee \neg w) \wedge (\top) \wedge (\bot \vee s \vee t) \wedge (\neg s \vee \neg t) \\
&= (x \vee y) \wedge (\neg x \vee y) \wedge (\neg y \vee \neg w) \wedge (s \vee t) \wedge (\neg s \vee \neg t) \qquad (5) \\
&= (x \vee y) \wedge (\neg x \vee y) \wedge (\neg y \vee \top) \wedge (s \vee t) \wedge (\neg s \vee \neg t) \\
&= (x \vee y) \wedge (\neg x \vee y) \wedge (s \vee t) \wedge (\neg s \vee \neg t) \\
&= (x \vee \top) \wedge (\neg x \vee \top) \wedge (s \vee t) \wedge (\neg s \vee \neg t) \\
&= (s \vee t) \wedge (\neg s \vee \neg t)
\end{aligned}$$

6. (80 pts.) Sudoku

Let's try out a real SAT solver, CryptoMiniSat. You can run it in the browser here, or see the README on its website for instructions on how to compile it on all major operating systems (or to run it in Docker).

Like essentially all SAT solvers, CryptoMiniSat reads CNF formulas in the DIMACS format, which has one clause per line:

```
p cnf 4 6
-1 -2 3 0
-1 4 0
1 -2 0
-2 -3 -4 0
1 2 0
2 -4 0
```

Clauses consist of a list of integer variable IDs (with a negative sign for negations), terminated by a 0: so the first clause in this example is $\neg x_1 \vee \neg x_2 \vee x_3$. The header line says that there are 4 variables and 6 clauses in total (in fact CryptoMiniSat is forgiving and you can omit this line).

(a) Describe a CNF encoding of the rules of Sudoku. (not assuming any cells are filled in)

Answer: let us use the variable $x_{r,c,v}$ to represent the value in the rth row, cth column and value v. 729 total variables.

i. every cell contains at least one number

$$\bigwedge_{r=1}^{9}\bigwedge_{c=1}^{9}\left(\bigvee_{v=1}^{9} x_{r,c,v}\right) \tag{6}$$

ii. every cell contains at most one number

$$\bigwedge_{r=1}^{9}\bigwedge_{c=1}^{9}\bigwedge_{v=1}^{8}\bigwedge_{i=v+1}^{9}\left(\neg x_{r,c,v} \vee \neg x_{r,c,i}\right) \tag{7}$$

the inside is equivalent to saying if one number, then not another number

iii. every number appears in a row at most once

$$\bigwedge_{r=1}^{9}\bigwedge_{v=1}^{9}\bigwedge_{c=1}^{8}\bigwedge_{i=c+1}^{9}\left(\neg x_{r,c,v} \vee \neg x_{r,i,v}\right) \tag{8}$$

iv. every number appears in a row at least once

$$\bigwedge_{r=1}^{9}\bigwedge_{v=1}^{9}\left(\bigvee_{i=1}^{9} x_{r,i,v}\right) \tag{9}$$

v. every number appears in a column at most once

$$\bigwedge_{c=1}^{9}\bigwedge_{v=1}^{9}\bigwedge_{r=1}^{8}\bigwedge_{i=c+1}^{9}\left(\neg x_{r,c,v} \vee \neg x_{i,c,v}\right) \tag{10}$$

vi. every number appears in a column at least once

$$\bigwedge_{c=1}^{9}\bigwedge_{v=1}^{9}\left(\bigvee_{i=1}^{9} x_{i,c,v}\right) \tag{11}$$

vii. the 3x3 restriction on values

$$\bigwedge_{v=1}^{9}\bigwedge_{b_x=0}^{2}\bigwedge_{b_y=0}^{2}\bigwedge_{x=1}^{3}\bigwedge_{y=1}^{3}\bigwedge_{(\alpha,\beta)>(x,y)}\left(\neg x_{3\cdot b_x+x,3\cdot b_y+y,v} \vee \neg x_{3\cdot b_x+\alpha,3\cdot b_y+\beta,v}\right) \tag{12}$$

the comparisons in the ordered pairs are lexographical.

(b) How would you use your encoding to solve a specific Sudoku puzzle? (i.e. where some of the cells are already filled in)

Answer: add single variable clauses. if a tile (a,b) had value v, then append clause $x_{a,b,v}$

(c) Use your encoding and CryptoMiniSat to solve "The World's Hardest Sudoku", shown below. You'll need to write a little program that either generates a DIMACS file or calls CryptoMiniSat's Python API (see here; alternatively, PySAT provides a Python interface to various other SAT solvers).

Figure 1: "The World's Hardest Sudoku", by Arto Inkala (accessed here).

Answer: (Please attach your code to the end of your PDF.)

```
from itertools import combinations

PUZZLE = [
    "800000000",
    "003600000",
    "070090200",
    "050007000",
    "000045700",
    "000100030",
    "001000068",
    "008500010",
    "090000400",
]
"""
solved

812 753 649
943 682 175
675 491 283

154 237 896
369 845 721
287 169 534

521 974 368
438 526 917
796 318 452

"""


def var(r, c, v):
    return 81 * r + 9 * c + v + 1


def main():
```

```
    i=input("decode")
    if i=="decode":
        grid = [[0] * 9 for _ in range(9)]
        for line in open("sol.txt"):
            if line.startswith("v "):
                for n in map(int, line.split()[1:]):
                    if 0 < n <= 729:
                        n -= 1
                        grid[n // 81][n // 9 % 9] = n % 9 + 1
        for row in grid:
            print("".join(map(str, row)))
        return

    clauses = []

    # each cell holds exactly one value
    for r in range(9):
        for c in range(9):
            clauses.append([var(r, c, v) for v in range(9)])
            for a, b in combinations(range(9), 2):
                clauses.append([-var(r, c, a), -var(r, c, b)])

    #each value appears exactly once in every row, column and box
    rows = [[(r, c) for c in range(9)] for r in range(9)]
    cols = [[(r, c) for r in range(9)] for c in range(9)]
    #3x3s
    boxes = [[(3 * br + i, 3 * bc + j) for i in range(3) for j in range(3)]
            for br in range(3) for bc in range(3)]

    for group in rows + cols + boxes:
        for v in range(9):
            clauses.append([var(r, c, v) for r, c in group])
            for (r1, c1), (r2, c2) in combinations(group, 2):
                clauses.append([-var(r1, c1, v), -var(r2, c2, v)])

    # givens
    for r, row in enumerate(PUZZLE):
        for c, ch in enumerate(row):
            if ch != "0":
                clauses.append([var(r, c, int(ch) - 1)])

    with open("sudoku.cnf", "w") as f:
        f.write(f"p cnf 729 {len(clauses)}\n")
        for cl in clauses:
            f.write(" ".join(map(str, cl)) + " 0\n")
    print("done")

if __name__ == "__main__":
```

```
main()
```

7. (optional, bonus question: 0 pts.) Lower bounds for BDDs

(a) In class I mentioned a non-constructive counting argument showing that *some* Boolean function on n variables must have an exponentially-large BDD (no matter what variable order you use). Flesh out the argument.

(b) **(hard)** Let $P_m(x_1, \ldots, x_{m^2})$ be true if and only if the matrix

$$\begin{pmatrix} x_1 & \cdots & x_m \\ \vdots & \ddots & \vdots \\ x_{m^2-m+1} & \cdots & x_{m^2} \end{pmatrix}$$

is a permutation matrix (i.e. has exactly one 1 in each row and column). Prove that any BDD for P_m must be exponentially large.